

02285 AI and MAS, F26

Multiagent planning

Curriculum for week 6: Sections 11.5 *Planning and Acting in Non-deterministic Domains* and 17.1 *Properties of Multiagent Environments* of R&N 4ed.

Today's subjects:

- Online planning.
- Cooperative distributed problem solving.
- Multiagent planning.
- Conflict-based search.

Assumptions in classical planning

Assumptions in classical planning (PDDL):

1. Environments are **deterministic**.
2. Environments are **fully observable**.
3. Environments are **static** (no exogenous events).
4. Planning is **offline** (compute entire plan before starting to act).
5. Environments are **single-agent** (only one planning agent).

Today we will consider loosening assumptions 3, 4 and 5 (the most relevant to the programming project).

Offline planning vs online (re)planning

Offline planning: Compute entire plan before executing first action (plan first, act later).

- **Ideal** when the domain is: **deterministic**, **static** and **fully observable**.
- **Problematic** in all other cases: one has to plan for **all contingencies** which easily leads to a combinatorial explosion.

Example. Box and goal locations in hospital domain not known in advance. Or plans of other agents not known in advance.

A **better solution** is often to do **online (re)planning**: interleave planning and acting. Example: Get this one box into its goal and then worry about the other boxes afterwards.

Then it is not sufficient with a simple search/planning algorithm, but we need to embed the planning algorithm into a larger **sense-plan-act** agent loop.

The necessity of replanning

Online replanning is in some cases our only option:

- The agent's model of the world (action schemas) might have **missing effects** or even **incorrect effects**.
- It can be impossible for the agent to model all possible **exogenous events** and their effects, including the agents performed by other agents.



Execution monitoring

Execution monitoring: Monitoring the execution of a computed plan to decide whether **replanning** is required.

Three levels of **execution monitoring** (from simple to more advanced):

- **Action monitoring:** *Does my next action still make sense?* When an action is about to be performed, check that all preconditions of the action still hold (e.g. by **sensing**). If not, replan to get to a state where original plan can be continued.
- **Plan monitoring:** *Does my existing plan still make sense?* When an action is about to be performed, check that the remaining plan will still succeed (nothing has destroyed the possibility of carrying out the remaining actions of the plan, and it will still lead to the goal). Partial-order planning can work well here.
- **Goal monitoring:** *Does my current goal still make sense?* When an action is about to be performed, check to see if there is a better set of goals that the agent might try to achieve. This requires some degree of **intentionality**, which will be discussed later.

Online replanning

Action monitoring is sometimes too naive.



Link to movie:

http://www2.compute.dtu.dk/~tobo/sphex_wasp_nosound.mov

4x speed

Link to movie:
<http://www2.compute.dtu.dk/~tobo/palm-e-planning-4x.mp4>

106

Multibody and multiagent planning

Multibody planning: A single agent planning for several acting entities. E.g. using best-first search in the space of joint actions to solve the MAPF problem in the hospital domain.

Multiagent planning: Several independent planning agents (several autonomous, proactive, goal-directed entities). E.g. letting each agent in the hospital domain plan for itself.

Goals in multiagent planning: Can be **individual** (independent of the other agents), **common** (e.g. players on the same team) or **conflicting** (e.g. players on opposing teams). This course: mostly common.

Agents sharing the same environment (resources) will most often need to **cooperate**, **coordinate**, and **negotiate** with each other. Compare with human social interaction.

Multibody vs multiagent planning

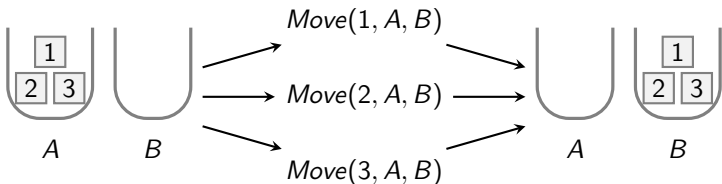
Multiagent planning can lead to a number of non-trivial challenges: communication, coordination, negotiation, deadlock handling, etc.

So isn't a centralised system (multibody planning) the better choice?

No, already the branching factor of multibody planning grows exponentially with $\#agents$. Multibody planning is in the space of **joint actions** $\langle a_1, \dots, a_n \rangle$. If the branching factor of each agent (no. of appl. actions) is b , the branching factor of the problem becomes b^n .

Some single-agent planning techniques can be adopted to multiagent without necessarily getting the (super)exponential blowup. E.g. POP planning can work well in loosely coupled multiagent problems. **Why?**

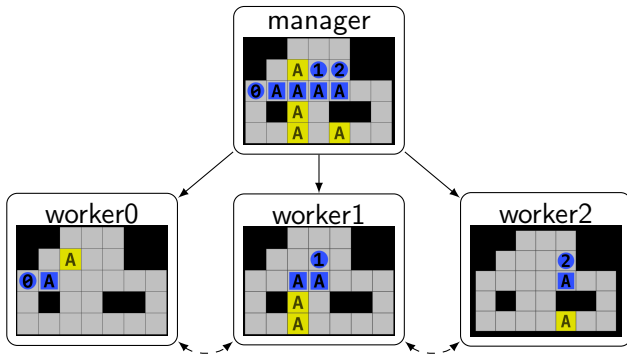
Different agents can be assigned to parallel paths in a partial-order plan.



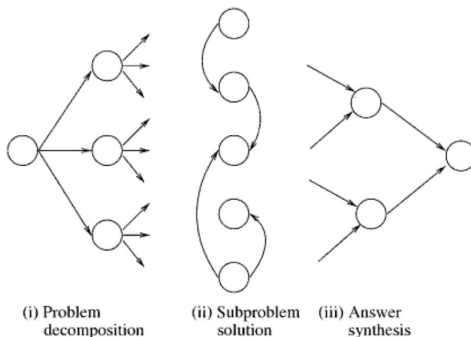
Multiagent architectures

Multiagent architecture: The arrangement of the agents and their protocols of interaction and collaboration in a multiagent system.

Example: A single *manager agent* and some *worker agents*. The manager distributes subgoals to the worker agents that compute their own individual plans. The architecture could include a communication protocol, e.g. allowing workers to require help from each other.



Cooperative distributed problem solving (CDPS)



Three-stage activity of CDPS agents:

- **Problem decomposition.** Decompose the problem into subproblems. Possibly hierarchically as in HTNs.
- **Subproblem solution.** Solve the individual subproblems. This stage typically involves information sharing between agents.
- **Solution synthesis:** Solutions to individual subproblems are integrated into an overall solution.

CDPS: Task sharing and result sharing

Two main cooperative elements of CDPS that require coordination and/or communication:

- **Task sharing** as part of *problem decomposition* (who does what?).
- **Result sharing** as part of the *subproblem solution* (e.g. partial results).

Task sharing consists of:

- **Decomposition** of the problem into smaller subproblems.
- **Allocation** of subproblems to the individual agents.

Result sharing: Information (partial results etc.) is distributed. Either **proactively** (agents send information they believe to be of interest to others) or **reactively** (as responses to requests from other agents).

Example: The Martha project (1998)

Multiagent architecture (multiagent planning system) for dock-worker robots using cooperative distributed problem solving:

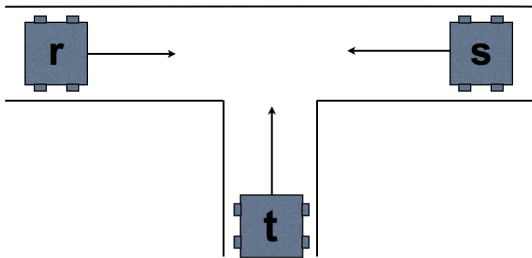
1. **Problem decomposition.** A **centralised planner** distributes transportation jobs to the individual dock-worker robots.
2. **Subproblem solution part 1.** The robots then plan and schedule their own transportation jobs **individually**, ignoring others.
3. **Subproblem solution part 2: Result sharing.** Each computed individual plan is **publicly broadcasted**.
4. **Subproblem solution part 3.** In case of **resource conflicts**, the robot causing it will be asked to replan, see next slide.



The Martha project (cont'd)

Resolving resource conflicts: When a robot r has computed a provisional plan (ignoring potential conflict with other agents), it broadcasts the plan. If a robot s is already committed to a plan that uses one of r 's resources, s will broadcast its own plan, and r will be required to replan to avoid the conflict.

Problem: r might not be able to replan, e.g. in case of a **deadlock loop**. In this case, the robot r will perform **centralised planning (multibody planning)** for the subset of robots involved in the conflict/deadlock loop.



Martha project (cont'd)

In the Martha project, the distribution of planning tasks reduces the complexity to **almost linear** in the number of robots (up to at least 50 robots in total).

Distributing planning tasks between agents is an example of **problem decomposition**. Relates to **partial-order planning** and **hierarchical planning**.

General approach underlying Martha project:

- First “pretend” the individual agents’ problems are completely decoupled (a **relaxed problem**).
- Solve the individual problems separately.
- Fix up the interactions (**conflicts**) by communication and coordination.

So the approach is: relax, solve, unrelax. Not completely unlike best-first search based on a heuristics derived from a relaxed problem. Also it’s the essential idea underlying the highly influential Conflict-Based Search algorithm that we will cover later in the course.

Partial global planning (Durfee, 1988)

Another multiagent architecture:

1. **Local plan formulation.** Each agent decides what its own goals are and generates short-term plans (local plans) in order to achieve them. “Short-term” means that the plans do not necessarily provide a sequence of actions leading *all* the way to the goals.
2. **Partial global plan construction and modification.** Local plans are integrated into a **partial global plan** capturing where local plans and goals interact. The partial global plan is modified to yield a more coordinated plan (e.g. avoiding redundancy and conflicts).
3. **Acting.** The partial global plan is translated back to the local level, and carried out by the individual agents.

Note relation to cooperative distributed problem solving (problem decomposition, subproblem solution, solution synthesis).

Independent and serialisable subgoals (Korf, 1987)

- **Independent subgoals.** Independent subgoals do not interfere with each other, and can be solved serially in any order: We can always achieve any additional subgoal without destroying the previous.
Example: Tidy up at home and buy groceries.
- **Serialisable subgoals.** Serialisable subgoals can be solved serially in *some order*: We can always achieve the next subgoal without destroying any previous. **Example:** Buy groceries and cook dinner.

Definition (Independent and serialisable subgoals, formal definition)

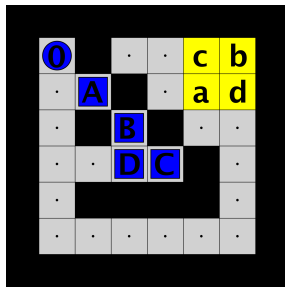
Suppose given a conjunctive goal $G = g_1 \wedge \dots \wedge g_n$ or set of subgoals $G = \{g_1, \dots, g_n\}$.

Independent subgoals: Let $G' \subseteq G$ and $g_i \in G \setminus G'$. Let S' be the states satisfying G' . Then from any state $s \in S'$ we can reach a state t satisfying g_i without ever leaving S' .

Serialisable subgoals: Suppose a correct order is $g_1, \dots, g_n$. Let S'_i be the set of states satisfying $\{g_1, \dots, g_i\}$. Then from any state $s \in S'_i$ we can reach a state t satisfying g_{i+1} without ever leaving S'_i .

Serialisable subgoals example

Here the subgoals are not independent. **Why not?** If we achieve the *A* and *D* goals first, we can't afterwards achieve *C* without destroying either *A* or *D*. But they are serialisable. **Why?** Solve *B* first, then no matter how, we can afterwards solve *C*, and no matter how, we can afterwards do *D*, and then finally *A*. We can't solve *A* before *D*, as agent might get stuck.



Trying SACrunch from the Warmup Assign.:

Solver	Time	Plan length	Nodes generated
<i>BFS</i>	297s	98	$\approx 9 \cdot 10^6$
<i>BFS</i> utilising ser. subgoals	0.25s	106	27.831
<i>A</i> *	0.86s	103	24.044
<i>A</i> * utilising ser. subgoals	0.12s	106	411
greedy	0.25s	164	2.792
greedy utilising ser. subgoals	0.10s	104	214

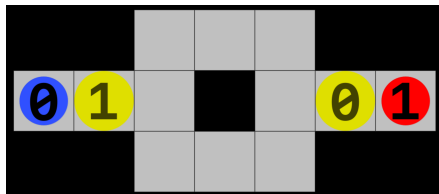
Non-serialisable subgoals

Definition (Non-serialisable subgoals)

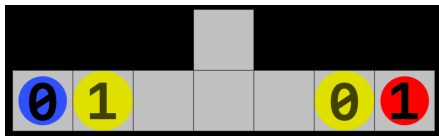
Subgoals not allowing serialisation, i.e., no order of subgoals exists satisfying the subgoal serialisability definition.

Sometimes it is still possible to perform a more advanced **merge** of the plans for the individual subgoals into one overall plan that achieves all subgoals. And sometimes not.

Plan merging: The scheduling/synchronisation of the individual plans (who gets to act when, in the case of multiagent planning).



Non-serialisable
some optimal solutions mergeable



Non-serialisable
optimal solutions not mergeable

Concurrent action constraints

Concurrent action constraints of an action a : Constraint on which actions has to be or must not be executed concurrently with a .

Example. Consider Gripper problem with several gripper arms (agents). When constructing **joint plans**, we must avoid several agents trying to move the same box at the same time.

ACTION: $Move(a, x, y, z)$

CONCURRENT: $\forall b \forall z' (b \neq a \rightarrow \neg Move(b, x, y, z'))$

PRECONDITION: $In(x, y)$

EFFECT: $In(x, z) \wedge \neg In(x, y)$

Or it might take two to lift a box:

ACTION: $Move(a, x, y, z)$

CONCURRENT: $\exists b (b \neq a \wedge Move(b, x, y, z))$

PRECONDITION: $In(x, y)$

EFFECT: $In(x, z) \wedge \neg In(x, y)$

How are concurrent action constraints specified in the Warmup

Assignment code? `is_conflicting`.

Plan merging

Suppose we are given a set of agents $A_1, \dots, A_n$, and that each agent A_i has produced its own plan $a_1^i, a_2^i, \dots, a_{m_i}^i$.

Now consider the following **joint plan**:

$$\begin{aligned} A_1 &: a_1^1, a_2^1, \dots, a_{m_1}^1 \\ A_2 &: a_1^2, a_2^2, \dots, a_{m_2}^2 \\ &\vdots \\ A_n &: a_1^n, a_2^n, \dots, a_{m_n}^n \end{aligned}$$

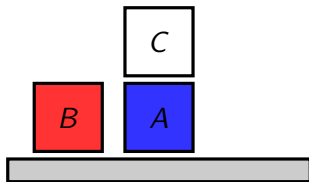
It is now possible to check:

1. Whether this constitutes a consistent joint plan, e.g. using the concurrent action lists of the involved actions
2. Whether it achieves all subgoals.

If item 1 fails, **conflicts** have to be resolved.

Plan merging example

Consider a blue and a red agent with individual goals $On(A, B)$ and $On(B, C)$ in the blocks world below (e.g. the blue agent can only move blue and white blocks, and the red only red and white):

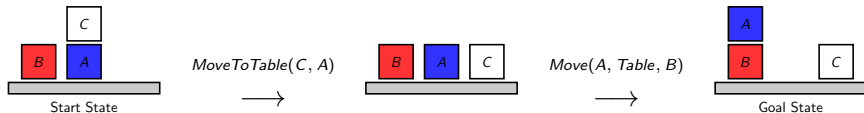


Start State

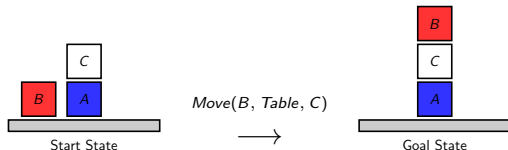
The two agents might come up with the individual plans shown on the next slide...

Plan merging example (cont'd)

Blue agent's plan for achieving $On(A, B)$:



Red agent's plan for achieving $On(B, C)$:

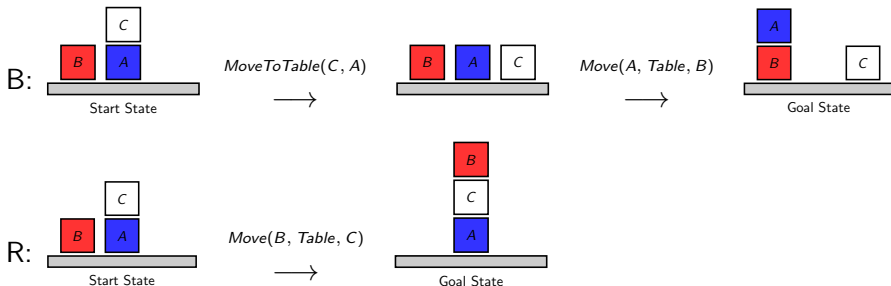


Thus the naive joint plan looks like this:

Blue : $MoveToTable(C, A), Move(A, Table, B)$

Red : $Move(B, Table, C)$

However, there are **conflicts**: block C cannot be moved while something is moved on top of it (by concurrent action list)...



Solution: Blue performs its first action before red's first. **Why not vice versa?** Red's first action destroys precondition for blue's first action.

After this, red performs its first action. **Why not blue's second action?** Destroys precondition for red's first action.

These **conflict resolutions** result in the following successful joint plan:

Blue : MoveToTable(C, A), NoOp, Move(A, Table, B)

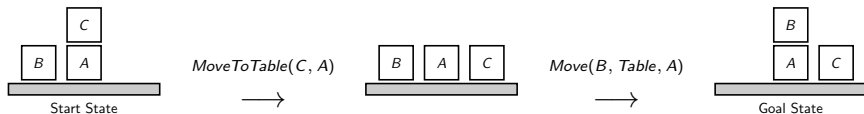
Red : NoOp, Move(B, Table, C), NoOp

It also doesn't work that blue carries out its entire plan before red starts or vice versa, since we don't have subgoal independence (cf. the *Sussman anomaly* in one of the exercise).

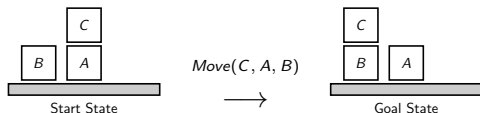
Plan merging example

Consider now instead individual goals $On(B, A)$ and $On(C, B)$ (with no constraints on who can move which blocks).

Blue's plan for achieving $On(B, A)$:



Red's plan for achieving $On(C, B)$:



Can these two individual plans be merged? No, each of the first actions of the two agents destroys the precondition for performing the other (at least in standard STRIPS formulation).